Runtime Systems: Execution, Data Representation & Memory

CG

November 7, 2026

Contents

1 Lazy evaluation **2**

1.1 Applicative and normal order . . . 2
1.2 Call by need . . . 3
1.3 Thunks . . . 4
1.4 Lazy lists . . . 5
1.5 Implementing lazy evaluation . . . 6
1.6 Evaluation to head normal form . . . 7
1.7 Exercises . . . 7

2 Runtime data representation **8**

2.1 Representation of code . . . 8
2.2 Run-time storage . . . 9
2.3 Representation of statements and data structures . . . 10
2.4 Objects and functions . . . 11
2.5 Homoiconicity . . . 12
2.6 Boxed and unboxed values . . . 14
2.7 Records, arrays and variants . . . 14
2.8 Closures and activation records . . . 15
2.9 Exercises . . . 16

3 Interpreters **17**

3.1 Tree-walking interpreters . . . 17
3.2 Eval and apply . . . 17
3.3 An interpreter for arithmetic, let and if . . . 17
3.4 Interpreters and compilers . . . 18
3.5 Metacircular interpreters . . . 18
3.6 Exercises . . . 19

4 Native code and intermediate representations **20**
4.1 Why an intermediate representation . . . 20
4.2 Three-address code . . . 20
4.3 Static single assignment . . . 21
4.4 From the intermediate representation to native code . . . 22
4.5 Exercises . . . 22

5 Virtual machines **23**
5.1 Stack machines and register machines . . . 23
5.2 The dispatch loop . . . 23
5.3 A trace . . . 24
5.4 Exercises . . . 24

6 Garbage collection **25**
6.1 Manual memory management . . . 25
6.2 Reference counting . . . 25
6.3 Mark and sweep . . . 26
6.4 The tri-color abstraction . . . 26
6.5 Copying collection . . . 27
6.6 Generational collection . . . 27
6.7 Exercises . . . 27

1 Lazy evaluation

The control-flow lecture evaluated the parameters of a function before passing them. This is not necessary: parameters can be passed unevaluated, and their evaluation delayed until the value is needed. This section follows Scott and Aldrich [7] and Watt [8].

1.1 Applicative and normal order

Definition 1.1. Applicative-order evaluation evaluates the parameters before evaluating the function. **Normal-order evaluation** does not evaluate the parameters when the function is invoked; it evaluates them when their values are needed.

Function evaluation is the reduction (simplification) of an expression into a value, through β-reduction steps. A fully evaluated expression is in normal form. For `square x = x * x`:

applicative order		normal order	
square $(3+4)$		*square* $(3+4)$	
$=$ *square* 7	def of $+$	$=(3+4)*(3+4)$	def of *square*
$=7*7$	def of *square*	$=7*(3+4)$	
$=49$	def of $*$	$=7*7=49$	

Applicative order is eager, strict evaluation (call by value). Normal order is non-strict: a function may return a result without evaluating an argument.

Theorem 1.2 (Church–Rosser)**.** *If two evaluation orders of an expression both terminate, they give the same result.*

This is the fundamental property of functional programming: the order does not change the value. It can change whether there is a value. With

```
three :: Integer -> Integer
three x = 3

infinity :: Integer
infinity = infinity + 1
```

normal order gives *three infinity* = 3 at once, because `three` never uses its argument. Applicative order evaluates `infinity` first and never ends: *three* (*infinity* + 1) = *three* ((*infinity* + 1) + 1) = ⋯ (Figure 1).

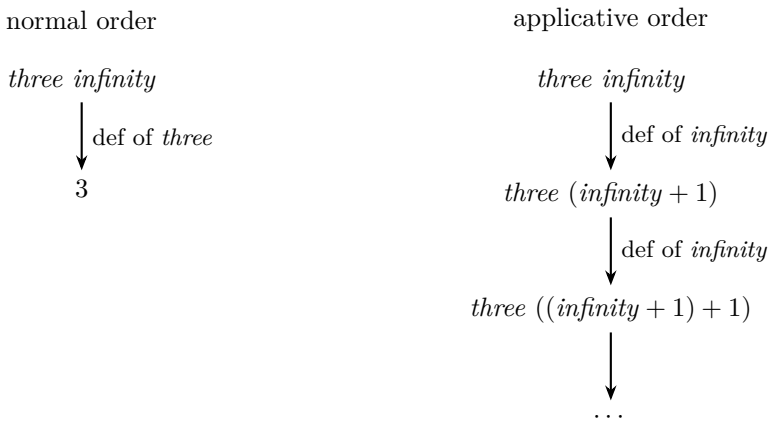


Figure 1: Reducing `three infinity` in the two orders.

In Scheme, which is applicative, the same effect needs an explicit promise:

```
(define (three x) 3)
(define (infinity) (+ (infinity) 1))

(define (three-lazy x) 3)
(define r (three-lazy (delay (infinity))))
```

Why lazy evaluation. The lecture lists its uses: avoiding unnecessary computation, manipulating streams, infinite and huge data structures, modularity (producer and consumer written separately), amortized costs, abstraction, and the freedom of evaluation order that pure code has, which allows parallelism, out-of-order optimization, asynchrony and reactive programming.

1.2 Call by need

Definition 1.3. Call by need is call by name in which, the first time an argument is evaluated, its value is stored (memoized) and reused in later uses.

The lecture's example is a function of two parameters whose body is `(a + b)/a`, called with `(3*2, 4/2)`. The parameters are bound to the unevaluated expressions $a \mapsto 3 * 2$ and $b \mapsto 4/2$. When the body needs `a` it evaluates `3*2`, gets 6, and replaces the expression by its value; when it needs `b` it evaluates `4/2`. The second use of `a`, in the denominator, finds 6 already computed. The result is $(6 + 2)/6$. (The slide calls the function `arithmetic_mean` and calls it as `add`; the body is not the arithmetic mean either.)

Two forms of non-strictness. Normal order and lazy evaluation are both non-strict, but they differ in sharing (Figure 2). In $(\lambda x.\, x * x)\,(3 + 4)$, normal order (call by name) copies the expression $3 + 4$ into each use of x and evaluates it twice. Lazy evaluation (call by need) keeps one copy of $3 + 4$ that both uses point to; when the first use evaluates it, the shared node becomes 7, and the second use reads 7. The key to lazy evaluation is to implement the evaluator with graph reduction.

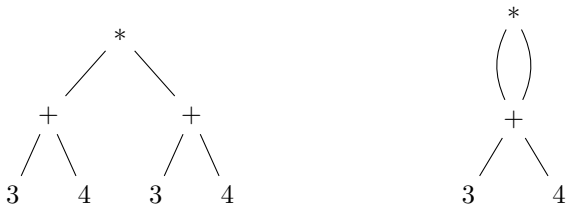


Figure 2: $x * x$ with $x = 3 + 4$: a tree with two copies (call by name, left) and a graph with one shared node (call by need, right).

The difference can be measured. A counter records how many times the argument is computed:

```
(define computed 0)
(define (arg) (set! computed (+ computed 1)) 7)

(define (square-value x) (* x x))
(define (square-name x) (* (x) (x)))
(define (square-need x) (* (force x) (force x)))

(define (count thunk)
  (set! computed 0)
  (let ((v (thunk))) (list v computed)))
```

The tests give 1 computation by value, 2 by name and 1 by need.

1.3 Thunks

Definition 1.4. A **thunk** is a parameterless function that suspends an expression, paired with the environment in which the expression occurs.

In Scheme, thunks are written by hand: `(delay <exp>)` or `(lambda () <exp>)` suspends the expression, and `(force <exp>)` or `(<exp>)` evaluates it. `delay` also memoizes the result; calling a `lambda` again computes it again. In Haskell thunks are the default strategy. A thunk is represented by the code to execute and its payload, the values of the free variables: for `sumSquare (3+4) (2*3)` with `sumSquare x y = x*x + y*y`, the thunk holds a pointer to the code of `sumSquare` and the two suspended arguments x and y (Figure 3). In C the same structure is a record with the code, the environment and a place for the value:

```
typedef struct Thunk {
    int evaluated;
    int (*code)(void *env);
    void *env;
    int value;
} Thunk;

int force(Thunk *t) {
    if (!t->evaluated) {
        t->value = t->code(t->env);
        t->evaluated = 1;
    }
    return t->value;
}
```

The test forces the thunk twice for `square (3+4)` and checks that the addition ran once.

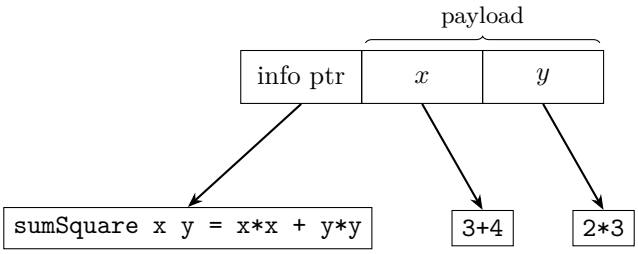


Figure 3: The thunk of `sumSquare (3+4) (2*3)`: a pointer to the code and the payload with the suspended arguments.

1.4 Lazy lists

Lazy evaluation builds data structures whose rest remains unevaluated. This makes infinite structures possible:

```
from :: Int -> [Int]
from n = n : from (n + 1)

isPrime :: Int -> Bool
isPrime n = n > 1 && all (\d -> n `mod` d /= 0) (takeWhile (\d -> d * d <= n) [2 ..])

firstPrime :: [Int] -> Int
firstPrime [] = 0
firstPrime (x:xs) = if isPrime x then x else firstPrime xs
```

`from 1` is $[1, 2, 3, \ldots]$, but after its first element is examined it is only `1 : from 2`, with the rest unevaluated. The lecture's exercise is to compute the first prime greater than m for any m: `firstPrime (from m)` examines `m`, `m+1`, ... until it finds a prime, and never builds the rest of the list. In Scheme, a lazy list is a pair whose second component is a promise:

```
(define-syntax lazy-cons
  (syntax-rules () ((_ a b) (cons a (delay b)))))
(define (lazy-car s) (car s))
```

```
(define (lazy-cdr s) (force (cdr s)))

(define (from n) (lazy-cons n (from (+ n 1))))

(define (take s k)
  (if (= k 0) '() (cons (lazy-car s) (take (lazy-cdr s) (- k 1)))))
```

```
(define (prime? n)
  (and (> n 1)
       (let loop ((d 2))
         (cond ((> (* d d) n) #t)
               ((= 0 (remainder n d)) #f)
               (else (loop (+ d 1)))))))

(define (first-prime s)
  (if (prime? (lazy-car s)) (lazy-car s) (first-prime (lazy-cdr s))))
```

Two other classic lazy lists are the infinite list of ones and the Fibonacci numbers defined in terms of themselves:

```
ones :: [Int]
ones = 1 : ones

fibs :: [Integer]
fibs = 0 : 1 : zipWith (+) fibs (tail fibs)
```

1.5 Implementing lazy evaluation

An evaluator evaluates expressions from the AST using a stack, from the bottom up. For `if x then if y then z else w` the inner `if` is an argument of the outer one, and lazy evaluation evaluates it only if `x` is true.

Graph reduction. A graph representation lets subexpressions share nodes, and the graph is reduced in the heap. For `square (square 3)`, with application nodes @, the argument `square 3` is one node that both uses of x in the outer `square` point to. Reducing the outer application gives $*$ with two edges to the same node `square 3`; when the first edge needs it, the node is reduced to 9 and overwritten, and the second edge sees 9. The result is $9 * 9 = 81$, with `square 3` computed once (Figure 4).

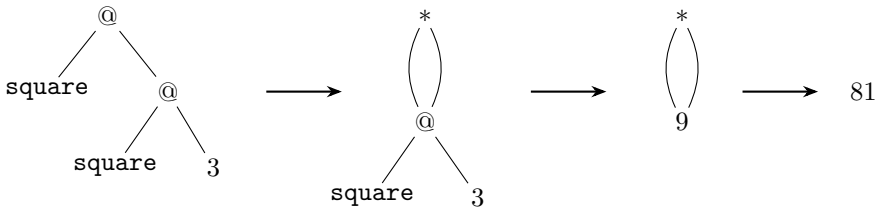


Figure 4: Lazy graph reduction of `square (square 3)`. The shared node `square 3` is reduced once.

Lazy graph reduction. The algorithm:

1. if the value is needed, find the top-level expression and reduce it;
2. if it is not needed, leave a reference to it and continue;
3. during evaluation, find the next redex (the expression to reduce), which is the leftmost one in the tree; reduce it; and update the root of that subgraph with the new value, so that every reference sees it.

Lazy evaluation is a transparent optimization: the programmer can think in terms of non-strict functions and normal-order evaluation, and the implementation avoids the cost of repeated evaluation.

1.6 Evaluation to head normal form

This subsection goes beyond the slides. Forcing a lazy value only evaluates it until its outermost constructor is known: `from 1` is forced to `1 : <thunk>`, not to a full list. This is weak head normal form (WHNF). Thunks can accumulate: `foldl (+) 0 [1..1000000]` builds a chain of a million suspended additions before computing any, a space leak. The strict version `foldl'` forces the accumulator at each step and runs in constant space; the test of `Lazy.hs` uses it. Compilers reduce these costs with strictness analysis, which finds the arguments a function always evaluates and passes them by value.

1.7 Exercises

Exercise 1.5. Compute the first prime greater than m, for any m.

Solution. `firstPrime (from m)`, with `firstPrime` as above. The list is infinite, but only the elements up to the first prime are evaluated. The tests give 97 for $m = 90$.

Exercise 1.6. Reduce `square (square 3)` in applicative order, normal order and with graph reduction. How many multiplications does each do?

Solution. Applicative: `square 3` $= 9$ first, then $9 * 9$: two multiplications. Normal order: $(\mathit{square}\ 3) * (\mathit{square}\ 3)$, each `square 3` computed separately: three multiplications. Graph reduction: the shared node is computed once: two multiplications, as in applicative order, while keeping non-strict semantics.

Exercise 1.7. Which result does `(lambda (x) 3)` give when applied to an expression that divides by zero, in Scheme and in Haskell?

Solution. Scheme evaluates the argument first and fails with a division by zero. Haskell never evaluates the argument, since the body does not use it, and returns 3.

2 Runtime data representation

Once a program has been processed, its information has to be stored somewhere. Variables are created in symbol tables, but their values, the code itself and the bookkeeping of the execution also need a place in memory. This section follows Pratt and Zelkowitz [6, Ch. 10], Aho et al. [1, Ch. 7] and Kaplan [4, Ch. 6, 9].

2.1 Representation of code

Code is kept in memory as text translated from the input program. A `case` statement

```
case expression of
   1:     clause_A
 | 2,7:   clause_B
 | 3..5:  clause_C
 | 10:    clause_D
 else     clause_E
end
```

is stored as a sequence of labelled blocks. To evaluate it, the translator generates a **jump table**, an array with the addresses of the points where the code continues, one entry per value between the smallest and the largest label:

```
      goto L6
L1:   clause_A
      goto L7
L2:   clause_B
      goto L7
L3:   clause_C
      goto L7
L4:   clause_D
      goto L7
L5:   clause_E
      goto L7
L6:   r1 := expression
      if r1 < 1 goto L5
      if r1 > 10 goto L5
      r1 := r1 - 1
      r2 := T[r1]
      goto *r2
L7:   ...
```

The table T has ten entries, for the values 1 to 10 (Figure 5). Values outside the range go to the `else` clause, and so do the values inside the range without a label (6, 8 and 9). The two tests at `L6` check the range; then the value, shifted to start at 0, indexes the table, and the jump goes to the address it finds. The slide writes the second test as `r1 > 1`, which would send every value except 1 to `clause_E`.

The same idea in C: the `switch`

```
char with_case(int e) {
    switch (e) {
    case 1: return 'A';
```

	T	
0	&L1	value 1: `clause_A`
1	&L2	values 2, 7: `clause_B`
2	&L3	values 3, 4, 5: `clause_C`
3	&L3	value 10: `clause_D`
4	&L3	values 6, 8, 9 and out of range: `clause_E`
5	&L5	
6	&L2	
7	&L5	
8	&L5	
9	&L4	

Figure 5: The jump table of the `case` statement, indexed by `r1 - 1`.

```
    case 2: case 7: return 'B';
    case 3: case 4: case 5: return 'C';
    case 10: return 'D';
    default: return 'E';
    }
}
```

and its version with an explicit table

```
static const char T[10] = {'A', 'B', 'C', 'C', 'C', 'E', 'B', 'E', 'E', 'D'};

char with_table(int e) {
    if (e < 1) return 'E';
    if (e > 10) return 'E';
    return T[e - 1];
}
```

give the same clause for every value from -3 to 14 in the tests.

2.2 Run-time storage

Everything a program needs at run time has to be stored:

1. the code segments, in translated form or in an intermediate representation (`x+y+z` becomes `r1 <- rx+ry`, `r2 <- r1+rz`);
2. system run-time programs: library functions (`sin`, `cos`), printing facilities, interpreters, translators and run-time storage functions;
3. user-defined data structures and constants (`final x = 3`, `global y = 0.1`);
4. sequence-control information: subprogram return points, coroutine resume points, event notices for scheduled subprograms;
5. temporary storage for the evaluation of expressions that cannot be kept in registers (`r1 = x+y`), and for the results of function calls when passing parameters (`return u+v`);

6. input-output buffers, used to store data to or from external storage;

7. system data: tables, status information for input-output, and state such as reference counts or garbage collection bits.

For a single user program these are laid out in four areas (Figure 6): the code area, the static area for data whose size and lifetime are known before execution, the stack for activation records, and the heap for data allocated and freed during execution. The stack and the heap grow toward each other into the free space between them.

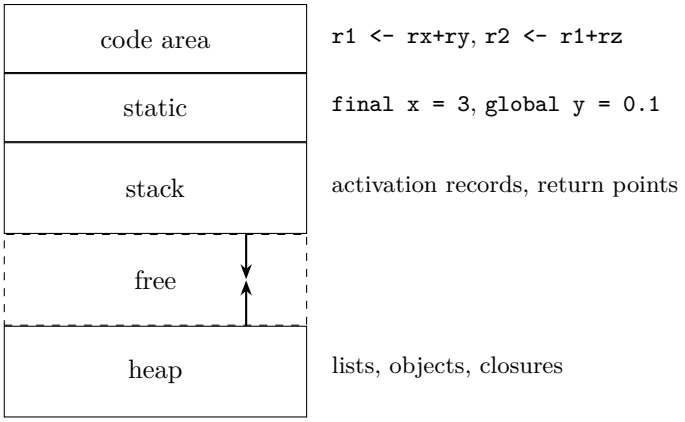


Figure 6: Memory of a single user program.

Storage management. At each execution step, memory is either allocated or free. Memory to allocate will be used by the program, and free memory is tracked at run time, normally with a **free-space list**. Used memory has to be recovered so that it becomes available again, either by moving the stack pointer back or by garbage collection (Section 6). As part of recovery, memory may need **compaction**: moving the blocks in use together so that the free memory forms larger blocks (Figure 7).

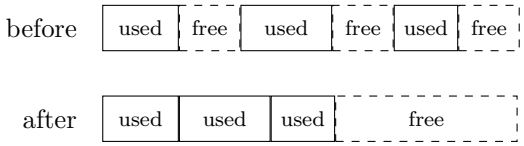


Figure 7: Compaction joins three small free blocks into one large block.

2.3 Representation of statements and data structures

Once translated, statements are represented in a linear data structure that records the position in memory of each statement, and a data structure, normally a list, that holds the statement: its type, its definition and a pointer to the next one.

Each data structure has its own representation, but most can be represented with records and lists. A **list** is a pointer to its first node, and each node holds a value and a pointer to the next node. A **record** $label(k_1 : v_1, \ldots, k_n : v_n)$ is represented in the same way: a pointer to a list of components, where each component is a pair whose key points to the corresponding value (Figure 8).

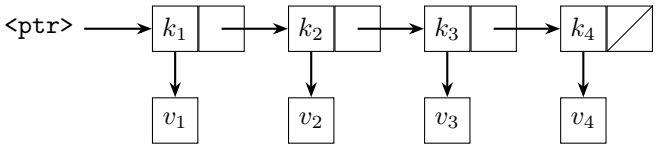


Figure 8: The record $label(k_1 : v_1, k_2 : v_2, k_3 : v_3, k_4 : v_4)$.

In Scheme a record of this kind is an association list:

```
(define point (list (cons 'x 3) (cons 'y 4)))
(define (field r k) (cdr (assq k r)))
```

2.4 Objects and functions

Following the same representation, other program elements such as objects and functions can be represented. The lecture's example is a class of linked lists with a global counter of instances:

```
int count = 0;

class list {
public:
    int info;
    list *next;
    list(int x) {
        info = x;
        count++;
        next = nullptr;
    }
    virtual int last() {
        if (next == nullptr) return info;
        else return next->last();
    }
};
```

An object of this class is a block with one cell for each field, `next` and `info`, and cells that point to the code of its methods, `last` and the constructor `list`. The counter `count` is not part of any object: it lives in the static area. The constructor code is also there, because it is the same for every object.

The second class question asks what happens with two instances. Each instance has its own `next` and `info`, but the pointer to the constructor has the same value in both, and should not be an independent copy: both objects point to the same code (Figure 9). The same holds for `last`. This is why compilers keep one table of methods per class and give each object a single pointer to it.

In Scheme an object can be written as a closure: the fields are local variables of the closure, the methods are the cases of a function that receives a message, and the static counter is a global variable. The tests build `lst1` and `lst2`, link them and check that `count` is 2 and that `last` of `lst1` is 2.

```
(define count 0)
(define (make-list-object x)
  (let ((info x) (next '()))
    (set! count (+ count 1))
```

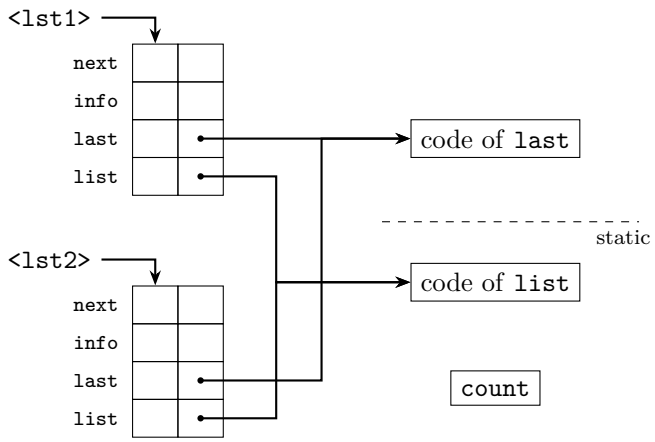


Figure 9: Two instances of `list`. Each has its own `next` and `info`; the method pointers have the same value in both.

```
(define (self message . args)
  (cond ((eq? message 'info) info)
        ((eq? message 'set-next!) (set! next (car args)))
        ((eq? message 'last)
         (if (null? next) info (next 'last)))))
self))
```

Functions. Creating a function $f = \textbf{fun}\ x_0 \ldots x_{k-1} \to e$ creates a function object F that holds the values of its free variables and the address of its code. In

```
(define (make-adder n)
  (lambda (x) (+ x n)))
(define add5 (make-adder 5))
(define add100 (make-adder 100))
```

`add5` and `add100` share the code `(+ x n)` and differ in the value of the free variable `n`.

Functions and parameters. A function allocates memory for its parameters by copying the values into memory, as dynamically allocated values, which can be used by all the scopes where they are visible. With `foo(x, y, z)` and `bar(v, w)`, the calls `foo(a, (b,c), (d,e))` and `bar((b,c), (d,e))` build the lists `(b,c)` and `(d,e)` in the heap. The parameters `y` and `v` point to the same list `(b,c)`, and `z` and `w` to the same list `(d,e)` (Figure 10).

2.5 Homoiconicity

Definition 2.1. A language is **homoiconic** when program code is represented with the language's fundamental data type.

In a homoiconic language code and data are intertwined: the program can build new code and manipulate it as data at run time. In Lisp the fundamental type is the list. The same text can be read as data or as code; quoting it tells Lisp not to evaluate it:

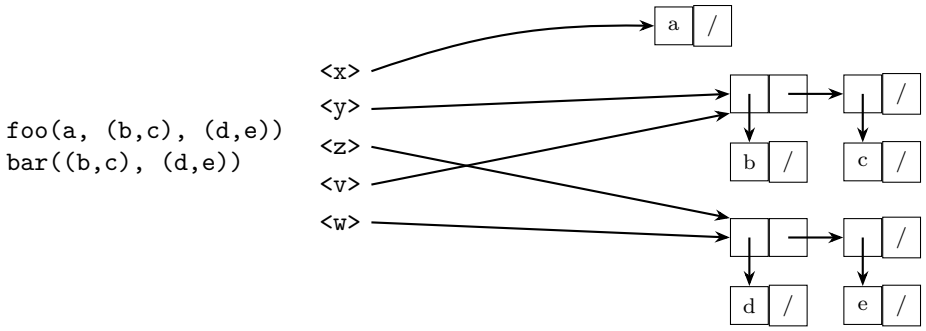


Figure 10: The parameters of `foo` and `bar` point to shared lists.

```
(list 1 2 (list 3 4))
(* 2 (+ 3 4))
'(* 2 (+ 3 4))
```

Figure 11 shows the two readings as trees. The lecture's example builds the expression $\sin(1.1) \cdot \cos(2.03)$ as a list, takes it apart, changes `cos` into `sin`, and evaluates the result:

```
(defvar expression (list '* (list 'sin 1.1) (list 'cos 2.03)))
(third expression)
(setf (first (third expression)) 'sin)
(eval expression)
(prin1-to-string expression)
```

The result is $\sin(1.1) \cdot \sin(2.03) \approx 0.79888$. The slide prints the expression with `print-to-string`; the standard Common Lisp function is `prin1-to-string`.

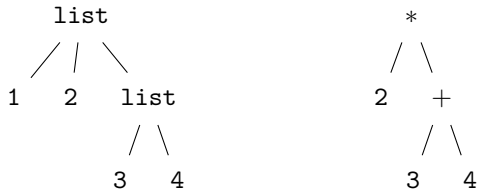


Figure 11: `(list 1 2 (list 3 4))` read as data and `(* 2 (+ 3 4))` read as code. Both are lists.

Scheme works the same way: a list that looks like a `lambda` becomes a function when it is evaluated.

```
(define square (lambda (x) (* x x)))
(define my-func (list 'lambda '(x) '(* x x)))
(define square2 (eval my-func (environment '(scheme base))))
```

The lecture also shows the idea in Prolog, where `clause(ten(X), Y)` returns the body `X is 2*5` of a clause as a term that can be inspected or called; in Julia, where `:(x + 1)` is a value of type `Expr` whose head and arguments can be changed before calling `eval`; and in Elixir, with `quote` and `Code.eval_quoted`.

Uniform representation. Haskell represents all its entities the same way: a header with an info pointer, followed by a payload. For a data value the payload is the data components, for a function value it is the free variables, and for a thunk it is the suspended arguments (Section 1).

Table 1 compares homoiconic and non-homoiconic languages.

Feature	Homoiconic	Non-homoiconic
Code and data	code is data, with the same structure	code and data are separate
Code representation	code is already an AST	code is parsed into an AST before it runs
Metaprogramming	natural	requires tooling such as reflection or code generation
Flexibility	DSLs, macros and self-modifying code are easy to write	static, the syntax is fixed

Table 1: Homoiconic and non-homoiconic languages.

2.6 Boxed and unboxed values

This subsection and the following ones go beyond the slides. A statically typed compiled language knows the type of every value at compile time, so a machine word holding an `int` contains only the bits of the integer. A dynamically typed language has to answer at run time which kind of value a bit pattern is. The answer is a **tag**, a small piece of metadata stored with the value or packed into it.

Definition 2.2. A value is **boxed** when it is represented as a pointer to a heap object that contains the tag and the data. It is **unboxed** (immediate) when the value, tag included, fits in a machine word.

Many Lisp and Scheme implementations keep small integers unboxed with tagged pointers. A pointer to a heap object is word-aligned, so its lowest bits are always zero. The lowest bit can then mark a small integer, with the value in the remaining bits (Figure 12). Telling a small integer from a pointer takes one bitwise `and`; a value outside the range of small integers falls back to a boxed representation, with an allocation and a pointer dereference.

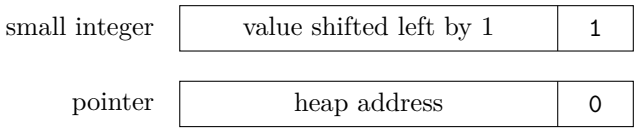


Figure 12: A tagged word: the lowest bit says whether the rest is an integer or an address.

2.7 Records, arrays and variants

In a statically typed language the layout of a record is fixed at compile time: each field has a byte offset from the start of the record, and reading `p.y` loads from the address of `p` plus the offset of `y`.

```
struct Point { double x; double y; };
```

A language whose objects can gain and lose fields at run time cannot fix the offsets. It stores the fields in a dictionary indexed by name, as the records of Figure 8, or caches the layout the first time it sees it.

An array stores elements of the same size contiguously, so indexing is arithmetic:

$$address(a[i]) = base(a) + i \times size(element).$$

A safe language checks $0 \le i < length$ before using the formula; C does not.

A value of a sum type, such as $\mathbf{List}\ \tau = \mu\alpha.\,\mathbf{Unit} + (\tau \times \alpha)$ from the recursive types section of the types notes, is a tag that says which variant it is (`Nil` or `Cons`) followed by the payload of that variant. Pattern matching reads the tag and jumps to the code for that variant.

2.8 Closures and activation records

A closure is a record with a pointer to the code of the function and a pointer to the environment with its free variables, the function object F described above. The Python version of a counter

```
def make_counter(start):
    n = start
    def bump():
        nonlocal n
        n += 1
        return n
    return bump
```

allocates one environment with `n` on each call to `make_counter`. The returned function always runs the same code against that environment, so `n` keeps its value between calls.

Each non-tail call needs an **activation record** (stack frame) for its local state, pushed on entry and popped on return. It holds the return address, the saved frame pointer of the caller, the parameters and the local variables. During `fact(3)`

```
int fact(int n) {
    if (n == 0) return 1;
    return n * fact(n - 1);
}
```

there are four frames on the stack when `fact(0)` is about to return (Figure 13). Each frame has its own `n`, which is why the recursive calls do not overwrite each other's variables.

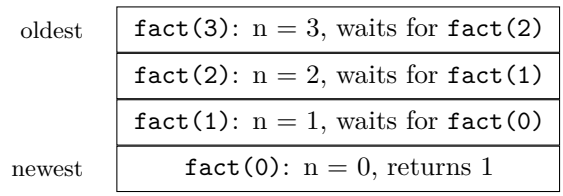


Figure 13: The stack during `fact(3)`.

A function nested inside another, as in Pascal, can use variables of the enclosing function, so it has to find the frame of that function. A **static link** is a pointer in each frame to the frame of the lexically enclosing function; reaching a variable k levels out follows k links. A **display** is an array indexed by nesting depth whose entry d points to the current frame at depth d; the access costs one array lookup, and every call and return has to update the array.

2.9 Exercises

Exercise 2.3. What happens if we add the element e_0 to the list $[e_1, e_2, e_3, e_4]$?

Solution. A new node is allocated with e_0 and a pointer to the node of e_1, and a new list pointer `<list_ptr'>` points to it (Figure 14). The old list is not copied: `<list_ptr>` still points to $[e_1, e_2, e_3, e_4]$, and both lists share its nodes. The slide's figure draws a fifth node e_5 after e_4, which is not part of the list.

```
(define list-ptr (list 'e1 'e2 'e3 'e4))
(define list-ptr2 (cons 'e0 list-ptr))
```

The tests check that the tail of the new list is `eq?` to the old list.

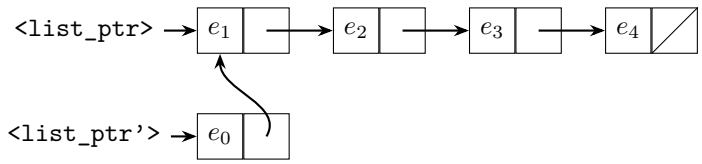


Figure 14: Adding e_0 allocates one node; the two lists share $e_1, \ldots, e_4$.

Exercise 2.4. Create two instances of the `list` object.

Solution. Each instance gets its own block with `next` and `info`. The cells for `list` and `last` hold the same values in both: the address of the same code. They should not be independent copies of the code (Figure 9). The global `count` is incremented by each constructor call, and ends at 2. Both the C++ and the Scheme versions check this.

Exercise 2.5. In Common Lisp, build the expression `(+ 1 (* 2 3))` as a list, replace `*` by `+`, and evaluate it.

Solution. The list is built with `(defvar e (list '+ 1 (list '* 2 3)))`. Evaluated now, it gives 7. The form `(setf (first (third e)) '+)` changes the inner list into `(+ 2 3)`, and `(eval e)` then gives 6. The end of `homoiconic.lisp` runs these steps.

Exercise 2.6. For the `case` statement of this section, which clause runs when `expression` is 8? Which table entry is read?

Solution. 8 is in the range 1 to 10, so the two tests fail and `r1` becomes 7; entry $T[7]$ is `&L5`, and `clause_E` runs, because 8 has no label.

3 Interpreters

This section and the following three go beyond the slides. They make concrete two ideas used throughout the course, the AST and the environment: an interpreter walks the AST with an environment and produces a result, without translating the program first. The presentation follows Friedman and Wand [2, Ch. 3] and Nystrom [5, Part II].

3.1 Tree-walking interpreters

A **tree-walking interpreter** takes the AST built by the parser and evaluates it recursively, node by node, according to the shape of each node. The tree that was parsed is the tree that runs. This is the simplest architecture to build, and the Rascal evaluators of the previous topics (the SIMP big-step semantics, the type checker of LETREC) have this shape.

3.2 Eval and apply

Most tree-walking interpreters are organized around two functions (Figure 15). `eval(expr, env)` looks at the shape of `expr`: a literal evaluates to itself, a variable is looked up in `env`, and a call evaluates the function and the arguments and passes the results to `apply`. `apply(f, args)` invokes a function value on evaluated arguments: it binds the parameters to the arguments in a new environment that extends the environment captured by the closure, and calls `eval` on the body in that environment.

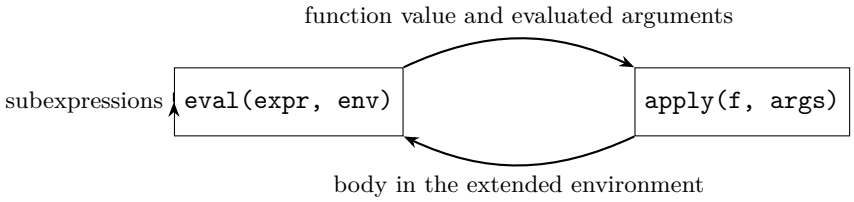


Figure 15: The eval/apply cycle.

Environments. In an interpreter the environment stops being the mathematical function $Var \to Val$ of the denotational semantics and becomes a data structure that the interpreter allocates, extends and searches. A common choice is a chain of frames: each scope has its own bindings and a pointer to the frame of the enclosing scope, and a lookup walks the chain outward. With a chain, a closure captures the current environment by keeping a pointer to the current frame.

3.3 An interpreter for arithmetic, let and if

The following interpreter handles integer arithmetic, `let` and `if`, with the AST written as Python tuples:

```
def evaluate(expr, env):
    tag = expr[0]
    if tag == 'num':
        return expr[1]
    elif tag == 'var':
```

```
        return env[expr[1]]
    elif tag == 'add':
        return evaluate(expr[1], env) + evaluate(expr[2], env)
    elif tag == 'mul':
        return evaluate(expr[1], env) * evaluate(expr[2], env)
    elif tag == 'let':
        _, name, e1, e2 = expr
        v1 = evaluate(e1, env)
        return evaluate(e2, {**env, name: v1})
    elif tag == 'if':
        _, cond, e1, e2 = expr
        if evaluate(cond, env) != 0:
            return evaluate(e1, env)
        else:
            return evaluate(e2, env)
    else:
        raise ValueError("unknown AST node: " + tag)
```

Example 3.1. The program `let x = 2 + 3 in if x then x * 10 else 0` is the tuple

```
program = ('let', 'x', ('add', ('num', 2), ('num', 3)),
           ('if', ('var', 'x'),
            ('mul', ('var', 'x'), ('num', 10)),
            ('num', 0)))
```

`evaluate` matches the `let` case and evaluates the bound expression: the two literals give 2 and 3, and the sum gives 5. It builds the environment $\{x \mapsto 5\}$ and evaluates the `if` in it. The condition is a lookup that gives 5, which is not 0, so the `then` branch runs: $5 \times 10 = 50$. The `else` branch is never evaluated. The tree of calls has the same shape as the AST.

3.4 Interpreters and compilers

Execution strategies form a range. A tree-walking interpreter evaluates the AST directly, and dispatches on the shape of each node every time the node runs, even inside a loop. A bytecode interpreter translates the AST once into a flat sequence of instructions and runs a dispatch loop over it (Section 5). A native compiler translates to the instructions of the machine, and no interpreter loop runs at execution time (Section 4). One implementation can use several strategies: the JVM interprets bytecode and compiles the methods that run often. "Interpreted" and "compiled" describe implementations, not languages.

3.5 Metacircular interpreters

A **metacircular interpreter** is an interpreter for a language written in the same language. The original Lisp `eval` was one. A small version in Scheme, for numbers, variables, `if`, `lambda` and application, with environments as lists of pairs:

```
(define (m-eval expr env)
  (cond ((number? expr) expr)
        ((symbol? expr) (cdr (assq expr env)))
        ((eq? (car expr) 'if)
         (if (m-eval (cadr expr) env)
```

```
            (m-eval (caddr expr) env)
            (m-eval (cadddr expr) env)))
        ((eq? (car expr) 'lambda)
         (list 'closure (cadr expr) (caddr expr) env))
        (else
         (m-apply (m-eval (car expr) env)
                  (map (lambda (a) (m-eval a env)) (cdr expr))))))

(define (m-apply f args)
  (if (procedure? f)
      (apply f args)
      (let ((params (cadr f)) (body (caddr f)) (env (cadddr f)))
        (m-eval body (append (map cons params args) env)))))
```

The functions are called `m-eval` and `m-apply` so that they do not replace the `eval` and `apply` of Scheme. The primitives come from the host language:

```
(define global-env
  (list (cons '+ +) (cons '* *) (cons '- -) (cons '< <)))
```

A `lambda` evaluates to a list that holds the parameters, the body and the environment of its definition, the function object F of Section 2. The last test applies a function defined where $x = 10$ inside a scope where $x = 99$, and gets 10: the interpreter implements static scope because `m-apply` extends the environment saved in the closure, not the environment of the call. Writing a metacircular interpreter forces every rule of the language to be explicit: what evaluates to itself, what a closure keeps, how application binds the arguments.

3.6 Exercises

Exercise 3.2. Trace the calls of `evaluate` for `let x = 4 in let y = x * x in y + 1`.

Solution. The outer `let` evaluates `4` and builds $\{x \mapsto 4\}$. The inner `let` evaluates `x * x` in it: two lookups and a product, 16, and builds $\{x \mapsto 4, y \mapsto 16\}$. The body looks up y and adds 1. The result is 17, which the tests check.

Exercise 3.3. The `let` case builds `{**env, name: v1}`, a copy of the environment. What does a chain of n nested `let`s cost, and what changes with a chain of frames?

Solution. The k-th `let` copies an environment of $k - 1$ bindings, so the chain costs $O(n^2)$ to build, and each lookup is $O(1)$. With frames each `let` adds one frame in $O(1)$, the chain costs $O(n)$, and a lookup may walk $O(n)$ frames.

Exercise 3.4. Change `m-apply` so that it extends the environment of the call instead of the environment of the closure. Which result does the last test give?

Solution. `m-apply` needs the environment of the call as a third argument, and `m-eval` passes its `env`. The body `(+ x y)` then finds the binding $x = 99$ first, and the result is 99: the interpreter implements dynamic scope, as in the `defvar` example of the scoping section of the semantics notes.

4 Native code and intermediate representations

A compiler can translate the AST directly into machine instructions, and simple compilers do. Most compilers translate first into an **intermediate representation** (IR). The jump table of Section 2, with its registers `r1`, `r2` and labels, is already written in an IR of this kind. This section follows Aho et al. [1, Ch. 6, 8].

4.1 Why an intermediate representation

The AST has the shape of the source language: `if` and `while` nodes and nested expressions. Machine code has the shape of the hardware: registers, flags, jump addresses. An IR is lower than the AST, with one operation per instruction and explicit temporaries, but it does not depend on a particular machine. Two things follow. Optimizations are written once, on the IR, and serve every target machine. And supporting a new machine only needs a new back end from the IR to that machine; the front end (parsing, type checking, generation of IR, optimization) is reused.

4.2 Three-address code

In **three-address code** every instruction has at most one operator and three operands, usually two sources and a destination, and every intermediate result gets a name, a **temporary**. The assignment `x = a + b * c` has a product nested inside a sum; in three-address code the product is computed into a temporary first:

```
t1 = b * c
x  = a + t1
```

The translation of expressions is a recursion over the AST that emits the code of the operands before the operation:

```
OPS = {'add': '+', 'sub': '-', 'mul': '*'}

def tac(expr, code, counter, target=None):
    if expr[0] == 'var':
        return expr[1]
    left = tac(expr[1], code, counter)
    right = tac(expr[2], code, counter)
    if target is None:
        counter[0] += 1
        target = 't' + str(counter[0])
    code.append(target + ' = ' + left + ' ' + OPS[expr[0]] + ' ' + right)
    return target

def assign(name, expr):
    code = []
    tac(expr, code, [0], name)
    return code
```

Control structures disappear in three-address code, replaced by labels and jumps. The `if` statement `if (x > 10) { y = 1; } else { y = 2; }` and the loop `while (i < n) { s = s + i; i = i + 1; }` become

```
    t1 = x > 10
    if t1 goto L_then
    goto L_else
L_then:
    y = 1
    goto L_end
L_else:
    y = 2
L_end:
```

```
L_test:
    t1 = i < n
    if t1 goto L_body
    goto L_end
L_body:
    s = s + i
    i = i + 1
    goto L_test
L_end:
```

This is the same translation the control-flow lecture used to explain `goto`: structured statements are abbreviations of jumps.

4.3 Static single assignment

Definition 4.1. A program is in **static single assignment** (SSA) form when each variable is assigned in exactly one place in the text.

A variable that the source assigns several times gets a new numbered name in each assignment. When two paths that assign a variable differently merge, a ϕ function at the merge point chooses the value according to the path that was taken:

```
if (cond) goto L1
x = 1
goto L2
L1:
x = 2
L2:
y = x + 1
```

```
if (cond) goto L1
x1 = 1
goto L2
L1:
x2 = 2
L2:
x3 = phi(x1, x2)
y1 = x3 + 1
```

Figure 16 shows the program in SSA form as a control-flow graph, whose nodes are the basic blocks. Each use of a variable now refers to one assignment, so the question of where a value comes from has a syntactic answer. Analyses such as finding dead code become simpler: an assignment `x2 = ...` whose name never appears on the right of another instruction is dead.

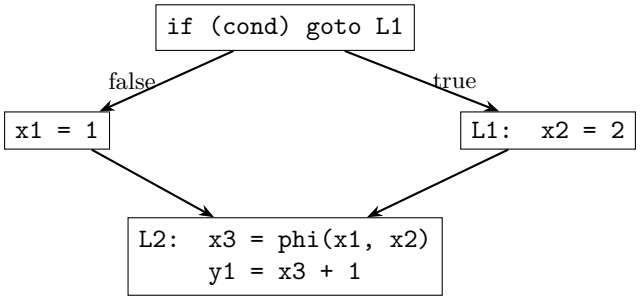


Figure 16: Control-flow graph in SSA form. The ϕ function takes `x1` on the left edge and `x2` on the right edge.

4.4 From the intermediate representation to native code

Instruction selection chooses, for each piece of IR, a sequence of machine instructions that computes the same thing at the lowest cost. One IR instruction can need several machine instructions, and some machines have one instruction for a common combination, such as a multiplication followed by an addition.

Register allocation maps the temporaries of the IR, as many as the translation needs, to the few registers of the machine. When there are not enough registers at some point, some temporaries are **spilled**: stored in the stack and loaded again when they are used. The classic method builds an *interference graph*, with a node per temporary and an edge between two temporaries that are live at the same point and cannot share a register. Allocating k registers is coloring this graph with k colors; a temporary that cannot be colored is spilled. The two problems are not independent: the best instruction can depend on how many registers are free, so compilers often interleave the two decisions.

4.5 Exercises

Exercise 4.2. Translate `z = (a + b) * (c - d)` into three-address code.

Solution. `t1 = a + b`, `t2 = c - d`, `z = t1 * t2`. The translator of this section produces exactly these three lines; the tests check it.

Exercise 4.3. Translate `if (a > b) { m = a; } else { m = b; }` into three-address code and then into SSA form.

Solution.

```
    t1 = a > b
    if t1 goto L_then
    m = b
    goto L_end
L_then:
    m = a
L_end:
```

```
    t1 = a > b
    if t1 goto L_then
    m1 = b
    goto L_end
L_then:
    m2 = a
L_end:
    m3 = phi(m2, m1)
```

The ϕ function takes `m2` when control arrives from `L_then`, that is, when $a > b$, and `m1` otherwise.

Exercise 4.4. A machine has three registers and, at some point, four temporaries are live. What does the register allocator do, and what does it cost?

Solution. The interference graph has four nodes joined to each other at that point, and needs four colors. One temporary is spilled: its value is stored in the stack and loaded again before each use. Each spill adds memory accesses, which are slower than register accesses.

5 Virtual machines

A tree-walking interpreter dispatches on every node each time it runs; a native compiler needs instruction selection, register allocation and a back end per machine. A bytecode **virtual machine** sits between the two: the AST is compiled once into the instructions of an abstract machine, and a simple interpreter runs those instructions. The abstract machine of SIMP in the semantics notes, with its control stack and its result stack, is a machine of this kind. This section follows Nystrom [5, Part III].

5.1 Stack machines and register machines

A **stack machine** (the JVM, the CPython virtual machine) keeps a stack of operands: each instruction takes its operands from the top of the stack and pushes its result. A **register machine** (the virtual machine of Lua since version 5) has virtual registers, and each instruction names its operands and its destination, as machine code does. For `(a + b) * c`:

```
push a
push b
add
push c
mul
```

```
add r0, a, b
mul r1, r0, c
```

The stack version has five simple instructions, and its compiler only walks the AST in postorder, emitting the operands before the operator. The register version has two instructions, but its compiler has to decide which register holds each value, a version of the register allocation problem of Section 4. The stack machine moves more data on and off the stack; the register machine needs a more complex compiler.

5.2 The dispatch loop

The core of the virtual machine is a **dispatch loop**: fetch the next instruction, select the code for it, run it, repeat.

```
void run(const unsigned char *ip, const double *constants) {
    for (;;) {
        unsigned char instruction = *ip++;
        switch (instruction) {
        case OP_CONSTANT: push(constants[*ip++]); break;
        case OP_ADD:      { double b = pop(), a = pop(); push(a + b); break; }
        case OP_SUBTRACT: { double b = pop(), a = pop(); push(a - b); break; }
        case OP_MULTIPLY: { double b = pop(), a = pop(); push(a * b); break; }
        case OP_PRINT:    printed = pop(); printf("%g\n", printed); break;
        case OP_RETURN:   return;
        }
    }
}
```

The `switch` is usually compiled to a jump table, like the one of Section 2, and every instruction goes back to the top of the loop. Faster virtual machines replace the central `switch` by *threaded*

code: each handler ends with a jump to the handler of the next instruction, taken from a table, without going back through the loop. In C this needs an extension of GCC and Clang (labels as values), so portable virtual machines keep the `switch`.

5.3 A trace

The program `print (2 + 3) * 4` compiles to

```
const double constants[] = {2, 3, 4};
const unsigned char code[] = {
    OP_CONSTANT, 0, OP_CONSTANT, 1, OP_ADD,
    OP_CONSTANT, 2, OP_MULTIPLY, OP_PRINT, OP_RETURN
};
```

where each `OP_CONSTANT` is followed by an index into the table of constants. Table 2 shows the stack after each instruction, and Figure 17 the same stack drawn at each step. Each row is one iteration of the loop.

Instruction	Effect	Stack after
`CONSTANT 2`	push 2	$[2]$
`CONSTANT 3`	push 3	$[2, 3]$
`ADD`	pop 3 and 2, push $2 + 3$	$[5]$
`CONSTANT 4`	push 4	$[5, 4]$
`MULTIPLY`	pop 4 and 5, push 5×4	$[20]$
`PRINT`	pop 20 and print it	$[\,]$

Table 2: Trace of `print (2 + 3) * 4`.

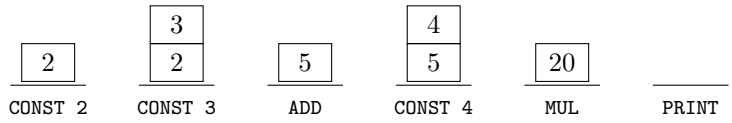


Figure 17: The operand stack after each instruction; the top is drawn highest.

5.4 Exercises

Exercise 5.1. Compile `(a - b) * (c + d)` for the stack machine and for the register machine.

Solution. Stack machine: `push a`, `push b`, `sub`, `push c`, `push d`, `add`, `mul`, seven instructions, with at most three values on the stack. Register machine: `sub r0, a, b`, `add r1, c, d`, `mul r2, r0, r1`, three instructions and three registers (two if `r2` reuses `r0`).

Exercise 5.2. Trace `CONSTANT 10`, `CONSTANT 4`, `SUBTRACT`, `CONSTANT 2`, `MULTIPLY`, `PRINT`.

Solution. The stack goes through $[10]$, $[10, 4]$, $[6]$, $[6, 2]$, $[12]$, $[\,]$, and 12 is printed. The second test of `vm.c` runs this program.

Exercise 5.3. Why can the compiler for the stack machine work with a postorder walk of the AST?

Solution. In postorder the code of the operands comes before the code of the operator. When the operator runs, its operands are the values on top of the stack, in order, and its result takes their place. The compiler never has to name where a value is.

6 Garbage collection

The storage management of Section 2 has to recover the memory that the program no longer uses: boxed values, closures, lists, objects. The stack is recovered by moving the stack pointer; the heap needs another mechanism. This section follows Jones, Hosking and Moss [3] and Scott and Aldrich [7].

6.1 Manual memory management

In C the programmer allocates with `malloc` and frees with `free`. Three errors are common. A **memory leak** is memory that the program can no longer reach and that was never freed. A **dangling pointer** is a pointer to memory already freed; using it reads or writes memory that may now belong to another object. A **double free** frees the same block twice and corrupts the bookkeeping of the allocator, such as the free-space list. **Garbage collection** removes the three errors: the run time decides when a value can no longer be used, and only then recovers its memory.

6.2 Reference counting

Each heap object keeps the number of references that point to it. Creating a reference increments the count, and destroying or overwriting one decrements it; when the count reaches zero, the object is freed at once. Recovery is incremental and immediate, with no pass over the whole heap. CPython uses reference counting as its main mechanism.

Reference counting does not recover cycles: objects that point to each other, with no reference from outside, keep their counts above zero.

```
class Node:
    def __init__(self):
        self.other = None

a = Node()
b = Node()
a.other = b
b.other = a
```

After

```
del a
del b
```

no name reaches the two objects, but each count is still 1, held by the field `other` of the other object. The test disables the cycle detector of CPython and checks with a weak reference that the objects survive; after `gc.collect()`, the detector has freed them. CPython pairs reference counting with this periodic detector.

6.3 Mark and sweep

A **tracing** collector asks which objects are reachable from the **root set**: global variables, the values in the stack and in the registers. **Mark and sweep** has two phases. The mark phase walks the graph of objects from the roots and marks every object it reaches. The sweep phase walks the heap and frees every object that was not marked. A cycle that is not reachable is never marked, and is freed with the rest (Figure 18).

The cost is **fragmentation**: the objects that survive stay where they were, with holes between them, and after many collections there may be no free block large enough for a new object even when the total free memory is enough. **Mark and compact** adds the compaction of Figure 7: it moves the live objects together and updates every pointer to a moved object.

6.4 The tri-color abstraction

During marking every object has one of three colors. *White* objects have not been reached; at the start every object is white, and at the end the white objects are garbage. *Gray* objects have been reached, but their pointers have not been followed yet. *Black* objects have been reached and all their pointers followed. Marking takes a gray object, colors gray the white objects it points to, and colors it black, until no gray object remains (Figure 18).

```
def mark(heap, roots):
    color = {obj: 'white' for obj in heap}
    gray = list(roots)
    for r in roots:
        color[r] = 'gray'
    while gray:
        obj = gray.pop()
        for target in heap[obj]:
            if color[target] == 'white':
                color[target] = 'gray'
                gray.append(target)
        color[obj] = 'black'
    return color

def sweep(heap, color):
    return {obj: refs for obj, refs in heap.items() if color[obj] == 'black'}
```

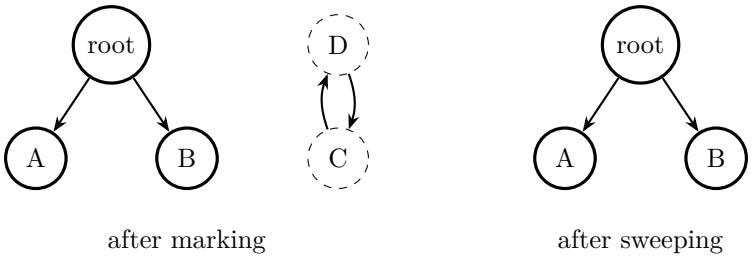


Figure 18: Mark and sweep. Black objects (thick) are reachable from the root; the cycle C, D stays white (dashed) and is freed.

The colors matter for collectors that run concurrently with the program, which in this context is called the **mutator**. If the mutator stores a pointer to a white object in a black object, which

will not be scanned again, the white object stays white and is freed while it is still in use. A **write barrier**, code inserted by the compiler in every store of a pointer, detects that case and colors the white object gray, so that it is scanned before the collection ends.

6.5 Copying collection

A **copying** collector divides the heap into two halves. The program allocates in one, *from-space*. When it is full, the collector copies the live objects into the other half, *to-space*, and the roles of the halves are exchanged. **Cheney's algorithm** does the copy without a stack: to-space itself is the queue of pending objects. A scan pointer walks to-space; for each object it copies the objects it points to that were not copied yet, at the end of to-space, until the scan pointer reaches the end.

```
def cheney(heap, roots):
    to_space = []
    for r in roots:
        if r not in to_space:
            to_space.append(r)
    scan = 0
    while scan < len(to_space):
        for target in heap[to_space[scan]]:
            if target not in to_space:
                to_space.append(target)
        scan += 1
    return to_space
```

The copied objects end up contiguous, so there is no fragmentation and no separate compaction. The cost of a collection is proportional to the live data, not to the size of the heap. The price is that half of the heap is always empty.

6.6 Generational collection

The **generational hypothesis** says that most objects die young: a temporary string, a list local to a loop, an intermediate result. A **generational** collector splits the heap into a young and an old generation. New objects go to the young generation, which is collected often: little of it is alive, so a collection is cheap and recovers much memory. An object that survives several collections is **promoted** to the old generation, which is collected rarely. Copying collection fits the young generation well, because its cost depends on the live data, which is small there.

6.7 Exercises

Exercise 6.1. Extend the cycle with a third object: `a.other = b`, `b.other = c`, `c.other = a`. What are the counts after the three names are deleted?

Solution. Each object has one reference, from the field `other` of the previous object in the cycle, so every count is 1 and reference counting frees none of them. The cycle detector or a tracing collector frees the three.

Exercise 6.2. A heap has a root pointing to A, A pointing to B, a pair C and D that point to each other, and an object E without references. Which objects are marked and which are swept?

Solution. The root, A and B are marked (black). C, D and E stay white and are swept. The tests of `collectors.py` check this heap.

Exercise 6.3. Trace Cheney's algorithm when from-space holds A, which points to B, B, and the garbage G, with A as the only root.

Solution. A is copied first, and the scan pointer is at A. Scanning A copies B; the scan pointer moves to B. B points to nothing, and the scan pointer reaches the end of to-space. To-space holds A, B in that order; G is never copied, and is lost when from-space is reused.

Exercise 6.4. Why collect the young generation more often instead of collecting the whole heap more often?

Solution. A full collection marks or copies all the live data, including the old objects that will survive anyway. A collection of the young generation only works on the young objects, most of which are already dead, so it recovers almost as much memory for a fraction of the work.

References

[1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2nd edition, 2007.

[2] Daniel P. Friedman and Mitchell Wand. *Essentials of Programming Languages*. The MIT Press, 3rd edition, 2008.

[3] Richard Jones, Antony Hosking, and Eliot Moss. *The Garbage Collection Handbook: The Art of Automatic Memory Management*. Chapman and Hall/CRC, 2011.

[4] Randy M. Kaplan. *Constructing Language Processors for Little Languages*. Wiley, 1994.

[5] Robert Nystrom. *Crafting Interpreters*. Genever Benning, 2021.

[6] Terrence W. Pratt and Marvin V. Zelkowitz. *Programming Languages: Design and Implementation*. Prentice Hall, 4th edition, 2001.

[7] Michael L. Scott and Jonathan Aldrich. *Programming Language Pragmatics*. Morgan Kaufmann, 5th edition, 2025.

[8] David A. Watt. *Programming Language Design Concepts*. Wiley, 2004.